AWS 3-Tier Architecture & Jenkins CI/CD Setup

Comprehensive Technical Deployment Guide & Step-by-Step Walkthrough

PART 1 Jenkins Server Setup & S3 Log Automation

1 Create EC2 Server Instance

Provision an Amazon EC2 instance (e.g., **jenkins-server** t2.micro). Ensure it is configured with appropriate security groups permitting SSH access and key pairs.

2 Install OpenJDK 17

Update system packages and install Java 21 runtime environment:

```
sudo apt update
sudo apt install openjdk-21-jre
java -version
```

3 Install Jenkins

Add the official Jenkins GPG key, repository list, and install Jenkins package:

```
curl -fsSL https://pkg.jenkins.io/debian/jenkins.io-2023.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null

echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt-get update
sudo apt-get install jenkins
```

4 Initial Jenkins Web Setup

Access the Jenkins Web UI via **http://<Public-IP>:8080**. Retrieve the initial administrative password from **/var/lib/jenkins/secrets/initialAdminPassword** to unlock the portal.

5 Plugin Configuration

Select **"Install Suggested Plugins"** to complete mandatory setup and launch the main dashboard.

PART 1 Jenkins Automation & S3 Log Backup (Cont.)

6 Jenkins Job Verification

Create your first Jenkins job (e.g., `First job`) and execute a test build to ensure execution capability.

7 S3 Storage Bucket Creation

In Amazon S3 console, create a general purpose bucket (e.g., `jenkinslogs-bucket`) in your target region for automated log archival.

8 Install & Configure AWS CLI

Install the AWS CLI v2 tools on your Jenkins instance and verify configuration:

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws --version

# Configure AWS Credentials
aws configure
```

10 Log Backup Script (`s3upload.sh`)

Create a bash shell script to parse daily Jenkins build logs and upload them to S3:

```
#!/bin/bash
JENKINS_HOME="/var/lib/jenkins"
S3_BUCKET="s3://jenkinslogs-bucket"
DATE=$(date +%Y-%m-%d)

if ! command -v aws &>/dev/null; then
    echo "AWS CLI is not installed. Exiting."
    exit 1
fi

for job_dir in "$JENKINS_HOME/jobs/"*/; do
    job_name=$(basename "$job_dir")
    for build_dir in "$job_dir/builds/"*/; do
        build_number=$(basename "$build_dir")
        log_file="$build_dir/log"
        if [ -f "$log_file" ] && [ "$(date -r "$log_file" +%Y-%m-%d)" == "$DATE" ]; then
            aws s3 cp "$log_file" "$S3_BUCKET/$job_name-$build_number.log" --only-show-errors
            [ $? -eq 0 ] && echo "Uploaded: $job_name/$build_number"
        fi
    done
done
```

Verification: Execute `chmod +x s3upload.sh && ./s3upload.sh` to confirm logs (`First job-1.log` , etc.) appear in the S3 bucket.

PART 2 AWS 3-Tier Architecture Deployment

We now build a production-grade 3-Tier Architecture on AWS comprising the **Web Tier (Presentation)**, **Application Tier (Logic)**, and **Database Tier (Data storage)**.

Web Tier Deployment

1 Create VPC & Subnets ("VPC and More")

Navigate to the VPC console and select **VPC and More** to automatically generate networking resources.

Setting	Configuration Value
Name Tag	**threeT**
IPv4 CIDR Block	**1 0.0.0.0/1 6**
Availability Zones	2 AZs (us-east-1 a, us-east-1 b)
Public Subnets	2 (Web Tier)
Private Subnets	4 (App Tier & DB Tier)
NAT Gateways	In 1 AZ (Cost-aware)
VPC Endpoints	S3 Gateway

2 Create Web Tier Launch Template

Configure instance template (**threeT-tmp**) under EC2 Launch Templates:

- **AMI:** Amazon Linux 2 AMI
- **Instance Type:** t2.micro (Free Tier eligible)
- **Security Group:** Create **threeTsec** (Allow SSH:22 and HTTP:80 from **0.0.0.0/0**)
- **Advanced Network:** Enable Auto-assign Public IP

User Data Script:

```
#! /bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
```

3 Web Tier Auto Scaling Group (ASG) & ALB

Configure Auto Scaling Group (`threeTAsg`) attached to template `threeT-tmp` :

- **Network:** Select VPC `threeT-vpc` and both Public Subnets.
- **Load Balancer:** Attach a new **Application Load Balancer** (`threeTLB`), scheme: **Internet-facing**.
- **Target Group:** Create a new target group.
- **Capacity Sizing:** Desired: 2, Minimum: 2, Maximum: 5.
- **Scaling Policy:** Target tracking scaling policy.

Testing Web Tier: Copy the public IP of an ASG EC2 instance into a browser. The default Apache HTTP Server test page indicates success.

Application Tier Deployment

1 Create Application Launch Template

Create template `threeT-tmp-app` using Amazon Linux 2 & t2.micro.

Security Rules (App Tier SG): Restrict access exclusively to the Web Tier Security Group:

- **Custom ICMP - IPv4:** Source = `Web-Tier-SG` (Allows ping tests)
- **SSH (22):** Source = `Web-Tier-SG`

2 Application Auto Scaling Group

- **ASG Name:** `threeT-asg-app`
- **Network:** Select VPC and 2 Private Subnets.
- **Load Balancer:** Attach internal ALB (`threeT-asg-app-1`), scheme: **Internal**.
- **Capacity:** Desired: 2, Min: 2, Max: 2.

3 Update Private Route Table

In VPC Route Tables, associate private subnets to ensure internal routing between App instances.

Database Tier Deployment

1 Create Database Subnet Group

In the Amazon RDS console, navigate to **Subnet Groups** and create `threeTsubnet-data` under `threeT-vpc`. Select Availability Zones (us-east-1 a, us-east-1 b) and associate the remaining two private subnets.

2 Provision MySQL RDS Database

Select **Standard Create** > **MySQL** (Engine Version 8.0.28):

Configuration	Setting
Template	Free Tier
DB Instance Identifier	`database-1`
Credentials	Username: `admin` | Master Password configured
Instance Class	`db.t2.micro` (Burstable)
Connectivity	Don't connect to EC2 | IPv4 | `threeT-vpc`
Public Access	**No** (Private database)
VPC Security Group	Create New (`threeTsec-data`)

3 Database Security Group Inbound Rules

Edit inbound rules for `threeTsec-data` : Set Type to **MySQL/Aurora (Port 3306)** and set Source to the **Application Tier Security Group** ID.

4 Connectivity Verification

Validate whole-architecture connectivity:

1. **Web Tier:** Public IP loads Apache web server successfully.
2. **App Tier Ping Test:** SSH into Web EC2 instance and ping Application EC2 private IP:

```
[ec2-user@ip-10-0-28-158 ~]$ ping 10.0.31.177
PING 10.0.31.177 (10.0.31.177) 56(84) bytes of data.
64 bytes from 10.0.31.177: icmp_seq=1 ttl=255 time=1.26 ms
64 bytes from 10.0.31.177: icmp_seq=2 ttl=255 time=0.97 ms
# Continuous packets confirm communication flow (Ctrl+C to stop)
```

Architecture Complete! The multi-tier infrastructure is now operational with isolated web, app, and data layer security controls.